PROCESSAMENTO DE ALTO DESEMPENHO

Otimização Sequencial, Hierarquia de Memória e Cache

Aula teórico-prática com Python

Curso	Engenharia de Computação
Disciplina	Processamento de Alto Desempenho
Semestre	2026.2
Nível	10º semestre — avançado
Foco	Desempenho sequencial, memória, cache e análise experimental

Ideia-chave: Antes de paralelizar, é preciso compreender o custo da versão sequencial. Em HPC, frequentemente o gargalo não é a quantidade de operações, mas a movimentação de dados entre os diferentes níveis de memória.

1. Objetivos de aprendizagem

- Analisar custo computacional e identificar oportunidades de otimização sequencial.
- Distinguir complexidade algorítmica de desempenho real medido.
- Reconhecer cálculos redundantes, alocações desnecessárias e sobrecarga de chamadas de funções.
- Compreender a hierarquia de memória: registradores, caches L1/L2/L3, RAM e armazenamento.
- Explicar localidade espacial e temporal, linhas de cache, cache hit e cache miss.
- Relacionar a ordem de percurso de matrizes ao layout físico dos dados na memória.
- Projetar benchmarks reproduzíveis e interpretar tempo, consumo de memória e speedup.
- Introduzir blocking/tiling como técnica de reorganização de acesso à memória.

2. Roteiro sugerido da aula

Bloco	Tempo sugerido	Atividade
A	35 min	O que significa otimizar? Custo, tempo, memória e metodologia de benchmark.
B	50 min	Práticas de otimização sequencial em Python.
C	40 min	Hierarquia de memória, cache, localidade e linhas de cache.
D	60 min	Experimentos com vetores e matrizes: linhas × colunas × stride.
E	35 min	Blocking/tiling, discussão, síntese e lançamento dos desafios.

3. Otimização sequencial

O objetivo da otimização sequencial é reduzir o tempo de execução e/ou o uso de recursos mantendo a correção do resultado. A otimização deve ser orientada por evidências: medir, localizar o gargalo, modificar, medir novamente e verificar se a saída continua correta.

> **Ideia-chave: Otimização prematura pode aumentar a complexidade do código sem produzir ganho relevante. Em engenharia de desempenho, a ordem correta é: medir → formular hipótese → modificar → validar → medir novamente.**

3.1 Métricas fundamentais

- Tempo de parede (wall-clock): tempo percebido entre início e fim.
- Tempo de CPU: tempo efetivamente gasto pela CPU no processo.
- Memória residente: quantidade de RAM ocupada durante a execução.
- Throughput: quantidade de trabalho concluída por unidade de tempo.
- Speedup: razão entre o tempo da versão de referência e o tempo da versão otimizada.
- Eficiência algorítmica: crescimento do custo em função do tamanho n da entrada.

Para duas versões que produzem o mesmo resultado, definimos o speedup sequencial como:

$$S = T_{referência} / T_{otimizada}$$

3.2 Como medir corretamente

Uma única medição é insuficiente. Processos do sistema operacional, aquecimento da CPU, gerenciamento de energia, compilação JIT de bibliotecas e efeitos de cache introduzem variabilidade.

- Usar time.perf_counter() para intervalos curtos.
- Executar várias repetições e reportar mediana; média e desvio-padrão podem ser adicionados.
- Separar tempo de preparação dos dados do trecho que se deseja medir.
- Manter a mesma entrada para todas as versões.
- Validar igualdade numérica dos resultados.
- Evitar print dentro da região medida.
- Fazer warm-up quando bibliotecas/rotinas tiverem custo inicial.

Código 1 — Função mínima de benchmark

```
from time import perf_counter
from statistics import median

def benchmark(func, *args, repeat=9):
    tempos = []
    resultado = None
    for _ in range(repeat):
        t0 = perf_counter()
        resultado = func(*args)
        tempos.append(perf_counter() - t0)
    return resultado, median(tempos), tempos
```

4. Técnicas de otimização sequencial

4.1 Redução de operações e eliminação de cálculos redundantes

Um cálculo que depende apenas de valores invariantes não deve ser repetido dentro de um laço. O ganho cresce com o número de iterações e com o custo da operação removida.

Código 2 — Loop-invariant code motion

```
# Versão A: recalcula uma constante n vezes
def versao_a(x, a, b):
    s = 0.0
    for xi in x:
        s += xi * ((a**2 + b**2) ** 0.5)
    return s

# Versão B: hoisting — move o invariante para fora do laço
def versao_b(x, a, b):
    c = (a**2 + b**2) ** 0.5
    s = 0.0
    for xi in x:
        s += xi * c
    return s
```

Discussão: A complexidade assintótica das duas versões continua O(n), mas a constante multiplicativa diminui. Big-O sozinho não descreve todo o desempenho.

4.2 Estruturas de dados

A escolha da estrutura afeta tanto o número de operações quanto a forma como os dados ocupam a memória. Um exemplo clássico é testar pertinência em list e set.

Código 3 — Estrutura de dados altera o custo da busca

```
import random

dados = list(range(1_000_000))
consultas = [random.randrange(1_200_000) for _ in range(50_000)]

def busca_lista(dados, consultas):
    return sum(q in dados for q in consultas)

def busca_set(dados, consultas):
    indice = set(dados)
    return sum(q in indice for q in consultas)
```

Em média, pertinência em uma lista exige busca linear O(n), enquanto uma tabela hash permite consulta média próxima de O(1). Entretanto, o set consome memória adicional: desempenho é uma troca entre recursos.

4.3 Custo de chamadas de função

Em Python, chamadas de funções têm overhead. Em tarefas de granulação extremamente fina, milhões de chamadas podem se tornar visíveis no perfil. Isso não significa que funções devam ser eliminadas indiscriminadamente: modularidade e clareza continuam importantes.

Código 4 — Microbenchmark de overhead de chamada

```
def quadrado(x):
    return x * x

def com_chamada(n):
    s = 0
    for i in range(n):
        s += quadrado(i)
    return s

def inline(n):
    s = 0
    for i in range(n):
        s += i * i
    return s
```

4.4 Alocação de memória

Criar e destruir objetos repetidamente custa tempo e pressiona o alocador de memória e o coletor de lixo. Pré-alocação é particularmente importante em arrays numéricos.

Código 5 — Crescimento dinâmico × pré-alocação

```
import numpy as np

def preencher_lista(n):
    y = []
    for i in range(n):
```

```
        y.append(i * i)
    return y

def preencher_array(n):
    y = np.empty(n, dtype=np.int64)
    for i in range(n):
        y[i] = i * i
    return y
```

Observação: em Python puro, list.append é altamente otimizado e usa crescimento amortizado. Portanto, o experimento deve ser interpretado com cuidado; a vantagem da pré-alocação aparece de forma mais clara em fluxos numéricos e estruturas de tamanho conhecido.

4.5 Otimizações de compilação

Em linguagens compiladas, níveis como -O2/-O3 permitem inlining, vetorização, eliminação de código morto e outras transformações. Em Python, o interpretador não aplica o mesmo conjunto de otimizações aos laços escritos em Python. Bibliotecas como NumPy deslocam operações para implementações compiladas, frequentemente em C/Fortran e com SIMD.

> **Ideia-chave: Comparar "Python puro" com NumPy não mede apenas cache: mede também interpretação, vetorização, bibliotecas compiladas e eventualmente multithreading interno. O experimento precisa deixar claro qual fenômeno está sendo investigado.**

4.6 Comparação entre algoritmos

A maior otimização costuma vir da escolha de um algoritmo melhor. Melhorar uma constante em $O(n^2)$ pode ser irrelevante diante da substituição por O(n log n).

Classe	$n=10^3$	$n=10^6$	Interpretação
O(1)	1	1	Constante
O(log n)	≈10	≈20	Crescimento lento
O(n)	1.000	1.000.000	Linear
O(n log n)	≈10.000	≈20.000.000	Quase linear
$O(n^2)$	1.000.000	10^{12}	Rapidamente proibitivo

5. Desafio 5 — Três versões sequenciais

Implementar três versões sequenciais de um mesmo algoritmo e comparar custo computacional, tempo e consumo de memória. A atividade deve mostrar não apenas qual versão é mais rápida, mas explicar por quê.

> **Entrega prevista no cronograma original: 9 de setembro.**

5.1 Requisitos mínimos

- Versão 1: implementação direta/de referência.
- Versão 2: eliminar cálculos redundantes, cópias ou alocações desnecessárias.
- Versão 3: alterar estratégia algorítmica ou estrutura de dados.
- Testar pelo menos cinco tamanhos de entrada.
- Executar pelo menos sete repetições por tamanho.
- Registrar mediana do tempo e memória máxima ou estimada.
- Demonstrar que as três versões produzem resultados equivalentes.
- Calcular speedup em relação à versão 1.

- Produzir gráfico tempo × tamanho da entrada.
- Explicar a relação entre análise teórica e resultado medido.

5.2 Sugestões de problemas

- Detecção de duplicatas: laços aninhados → list → set.
- Cálculo de distâncias entre pontos com remoção de expressões invariantes.
- Histograma ou contagem de frequências com diferentes estruturas.
- Busca de pares que satisfazem uma condição usando força bruta e indexação.
- Processamento de séries temporais com criação repetida de listas versus buffers/arrays.

6. Hierarquia de memória

Processadores modernos executam operações aritméticas muito mais rapidamente do que a memória principal consegue fornecer dados. Para reduzir essa diferença, o sistema utiliza vários níveis de armazenamento, pequenos e rápidos próximos à CPU e grandes e lentos mais distantes.

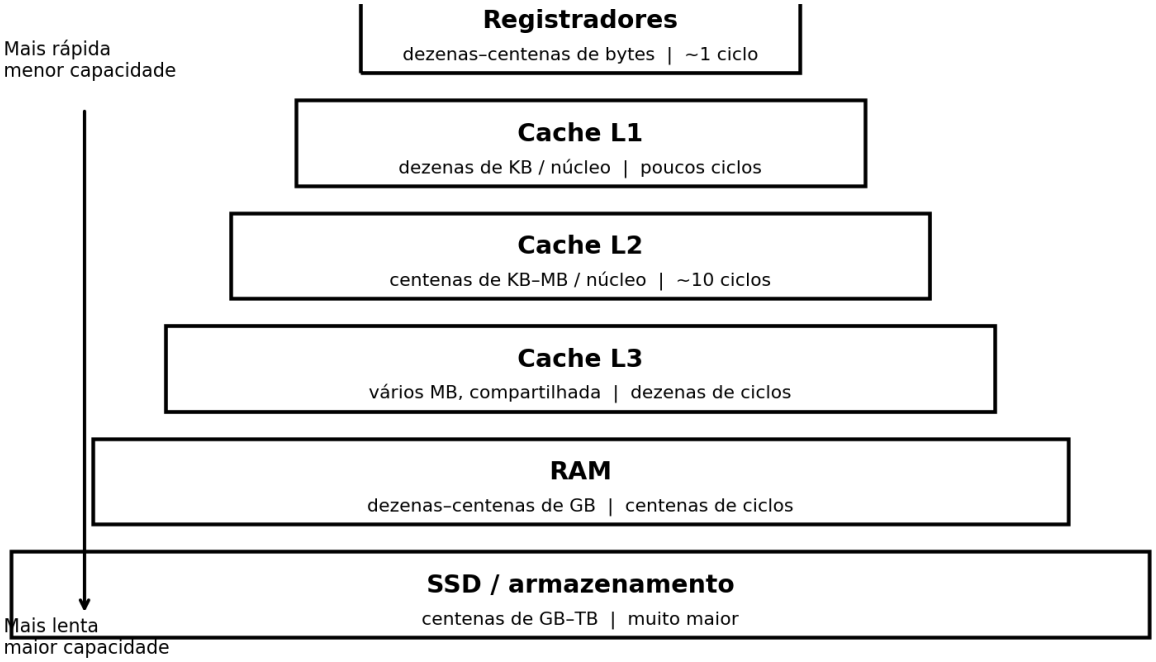


Figura 1 — Representação qualitativa da hierarquia de memória. Latências são ordens de grandeza aproximadas e variam entre arquiteturas.

> **Ideia-chave: Uma operação aritmética só pode ser executada quando seus operandos estão disponíveis. Assim, desempenho pode ser limitado pela capacidade de levar dados até as unidades de execução, e não pela velocidade da operação matemática.**

6.1 Cache L1, L2 e L3

- L1: menor e mais rápida; normalmente separa cache de dados e de instruções.
- L2: maior, um pouco mais lenta, frequentemente privada por núcleo.
- L3/LLC: ainda maior, mais lenta e frequentemente compartilhada entre núcleos.
- RAM: muito maior que as caches, porém com latência significativamente superior.

Detalhes de tamanho, associatividade e compartilhamento dependem do processador. Em uma análise experimental séria, os alunos devem consultar as especificações da CPU usada no laboratório.

6.2 Linhas de cache

A CPU não costuma transferir um único byte isoladamente da memória para a cache. Os dados são transferidos em blocos chamados linhas de cache. Em muitas arquiteturas atuais, 64 bytes é um tamanho comum, embora isso não deva ser assumido universalmente.

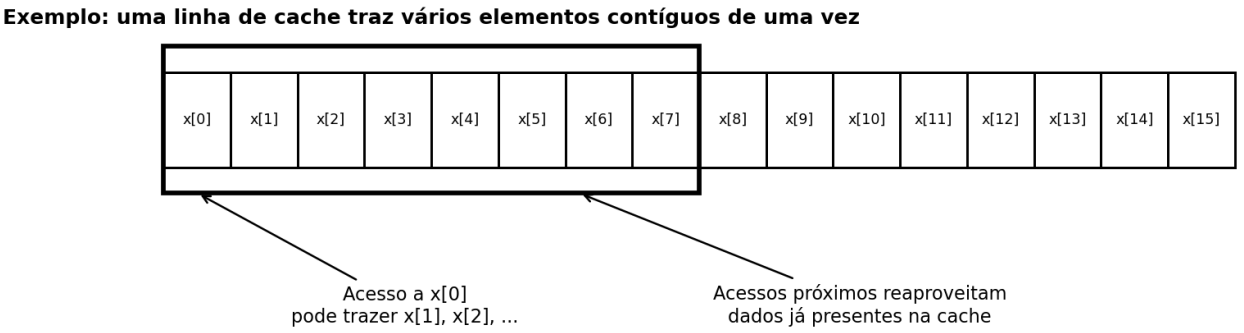


Figura 2 — Consequência da transferência em linhas: acessar dados próximos tende a aproveitar bytes já carregados.

Exemplo: para valores float64 de 8 bytes, uma linha de 64 bytes pode transportar 8 valores contíguos. Ao acessar x[i], dados de x[i+1], x[i+2] etc. podem já chegar à cache.

6.3 Localidade espacial e temporal

Princípio	Significado	Exemplo
Localidade temporal	Dados usados agora têm chance de ser usados novamente em breve.	Reutilizar coeficiente ou pequeno bloco várias vezes.
Localidade espacial	Após acessar um endereço, há chance de acessar endereços vizinhos.	Percorrer vetor sequencialmente.
Má localidade	Acessos distantes/irregulares reduzem reaproveitamento.	Saltar por grandes strides ou seguir índices aleatórios.

6.4 Cache hit e cache miss

Um cache hit ocorre quando o dado procurado já está no nível consultado. Um cache miss exige buscar o dado em nível inferior, aumentando a latência. Nem todo miss tem a mesma causa: há misses compulsórios (primeiro acesso), de capacidade e de conflito.

- Compulsório: primeira referência a uma linha de dados.
- Capacidade: conjunto ativo é maior do que a cache disponível.
- Conflito: diferentes endereços competem pelos mesmos conjuntos da cache, dependendo da associatividade.

7. Prática 1 — Acesso sequencial e stride

O objetivo é observar como a distância entre posições consecutivamente acessadas modifica o tempo por elemento. O experimento deve usar um array maior que as caches de níveis superiores.

Código 6 — Experimento de stride

```
import numpy as np
from time import perf_counter

N = 40_000_000
x = np.ones(N, dtype=np.float64)
```

```
def soma_stride(x, stride):
    s = 0.0
    for i in range(0, x.size, stride):
        s += x[i]
    return s

for stride in [1, 2, 4, 8, 16, 32, 64, 128, 256]:
    t0 = perf_counter()
    s = soma_stride(x, stride)
    dt = perf_counter() - t0
    acessos = (N + stride - 1) // stride
    print(stride, dt, dt / acessos * 1e9, 'ns/acesso', s)
```

Perguntas orientadoras:

- Por que comparar tempo total pode ser enganoso quando strides maiores fazem menos acessos?
- Qual métrica deve ser usada: tempo total, tempo por acesso ou ambos?
- Em que faixa o padrão muda de maneira mais perceptível?
- O prefetcher de hardware pode influenciar o resultado?
- Por que resultados diferem entre processadores?

8. Prática 2 — Percurso de matrizes e organização física

Por padrão, arrays criados normalmente pelo NumPy usam ordem C (row-major): elementos consecutivos de uma linha estão contíguos na memória. Portanto, para laços executados em Python/Numba/C, a ordem de percurso pode alterar significativamente a localidade espacial.

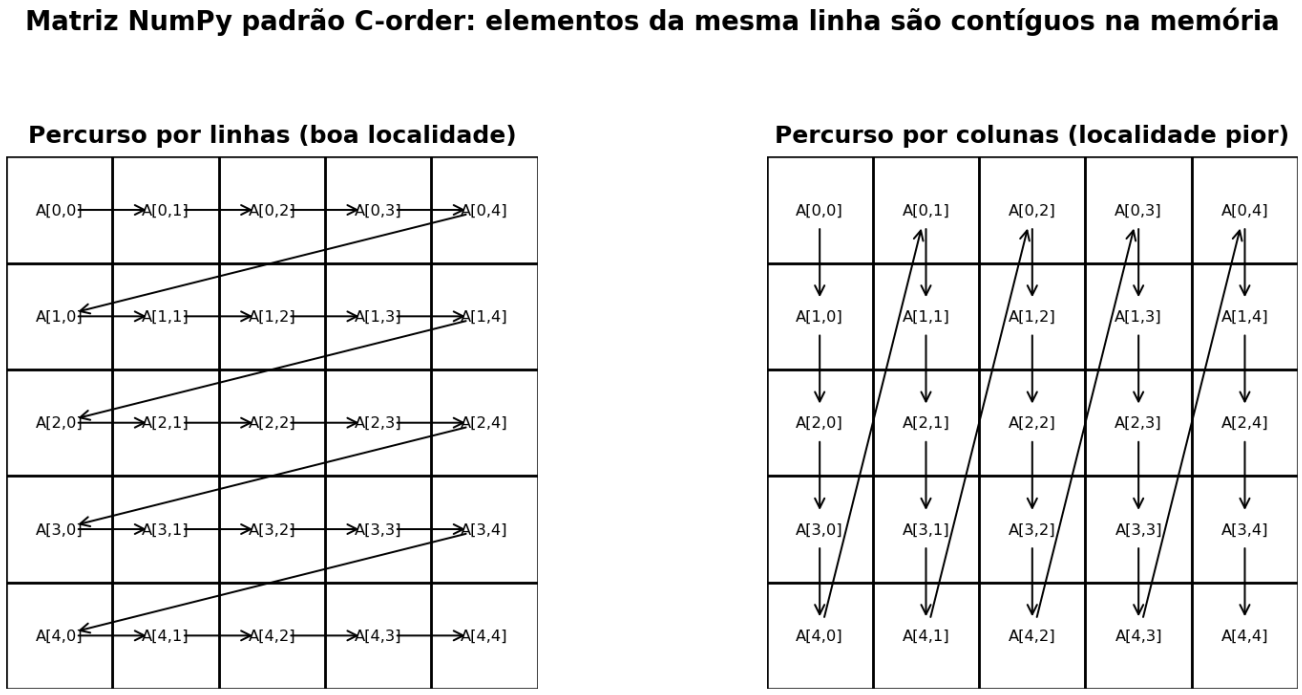


Figura 3 — Ordem lógica de acesso e continuidade física em uma matriz C-order.

Código 7 — Percurso por linhas × colunas

```
import numpy as np
from time import perf_counter

N = 3000
A = np.random.random((N, N))

def soma_linhas(A):
    s = 0.0
    n, m = A.shape
    for i in range(n):
        for j in range(m):
            s += A[i, j]
    return s

def soma_colunas(A):
    s = 0.0
    n, m = A.shape
    for j in range(m):
        for i in range(n):
            s += A[i, j]
    return s

for f in (soma_linhas, soma_colunas):
    t0 = perf_counter()
    r = f(A)
    print(f.__name__, perf_counter() - t0, r)
```

Ideia-chave: Em Python puro, o overhead do interpretador pode dominar o custo e mascarar parte do efeito de cache. Isso é pedagogicamente útil: mostra que um benchmark deve isolar o fenômeno investigado. Para observar cache com maior nitidez, repita usando Numba ou uma implementação compilada.

8.1 Versão com Numba (opcional, recomendada)

Código 8 — Removendo grande parte do overhead do interpretador

```
from numba import njit

@njit
def soma_linhas_jit(A):
    s = 0.0
    n, m = A.shape
    for i in range(n):
        for j in range(m):
            s += A[i, j]
    return s

@njit
def soma_colunas_jit(A):
    s = 0.0
    n, m = A.shape
    for j in range(m):
        for i in range(n):
            s += A[i, j]
    return s
```

```
# Warm-up: exclua o custo da compilação da medição
soma_linhas_jit(A); soma_colunas_jit(A)
```

Após o warm-up, medir várias repetições. Como a matriz está em C-order, espera-se vantagem para o percurso por linhas. Para uma matriz criada em Fortran-order, a situação tende a inverter-se.

Código 9 — Verificando layout C-order e F-order

```
AF = np.asfortranarray(A)
print(A.flags['C_CONTIGUOUS'], A.flags['F_CONTIGUOUS'])
print(AF.flags['C_CONTIGUOUS'], AF.flags['F_CONTIGUOUS'])
```

9. Prática 3 — Tamanho do conjunto de trabalho

Agora variamos o volume de dados. A ideia é verificar se o custo por elemento muda quando o conjunto de trabalho deixa de caber nos níveis superiores da cache.

Código 10 — Tempo por elemento × tamanho do conjunto de trabalho

```
import numpy as np
from time import perf_counter

for mib in [0.25, 0.5, 1, 2, 4, 8, 16, 32, 64, 128, 256]:
    n = int(mib * 1024**2 / 8)
    x = np.ones(n, dtype=np.float64)

    # warm-up
    _ = x.sum()

    tempos = []
    for _ in range(7):
        t0 = perf_counter()
        r = x.sum()
        tempos.append(perf_counter() - t0)

    med = np.median(tempos)
    print(f'{mib:7.2f} MiB  {med*1e3:8.3f} ms  {med/n*1e9:8.3f} ns/elem')
```

Interpretação: não se deve prometer que cada “degrau” corresponderá exatamente a L1/L2/L3. Prefetch, vetorização, bandwidth, política de substituição, frequência dinâmica e implementação de np.sum influenciam o resultado. O objetivo é formular hipóteses e compará-las com as especificações da CPU.

10. Blocking / tiling

Blocking divide o problema em blocos menores, buscando manter o conjunto de dados reutilizado dentro de um nível rápido da memória. É uma técnica fundamental em multiplicação de matrizes, processamento de imagens e métodos numéricos.

Blocking reorganiza o trabalho para reutilizar subconjuntos que cabem melhor na cache

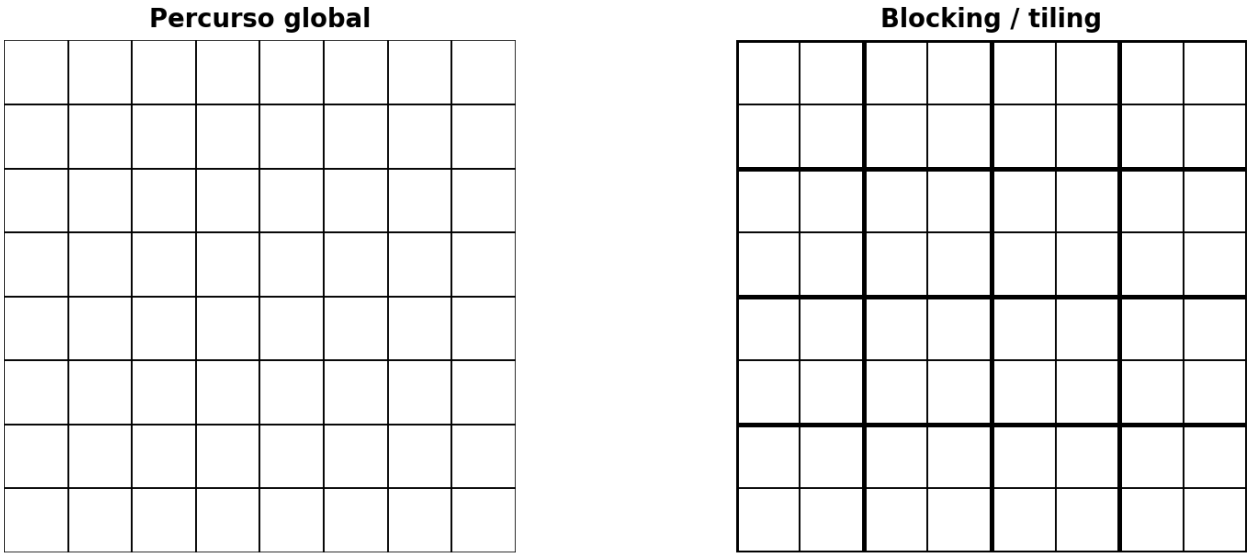


Figura 4 — Ideia de blocking: trabalhar repetidamente em regiões pequenas antes de avançar.

Na multiplicação C = A × B, uma implementação ingênua pode recarregar repetidamente partes de A e B. A versão blocada percorre submatrizes, aumentando reutilização temporal.

Código 11 — Estrutura conceitual de multiplicação blocada

```
def matmul_blocked(A, B, BS=64):
    n = A.shape[0]
    C = np.zeros((n, n), dtype=A.dtype)
    for ii in range(0, n, BS):
        for kk in range(0, n, BS):
            for jj in range(0, n, BS):
                i2 = min(ii + BS, n)
                k2 = min(kk + BS, n)
                j2 = min(jj + BS, n)
                for i in range(ii, i2):
                    for k in range(kk, k2):
                        aik = A[i, k]
                        for j in range(jj, j2):
                            C[i, j] += aik * B[k, j]
    return C
```

Este código é deliberadamente didático. Em Python puro será lento; seu valor é revelar a ordem de acesso. Para benchmarking sério, use Numba/C/C++ ou compare bibliotecas BLAS. O tamanho ótimo do bloco depende da arquitetura e do tipo de dado.

11. Python, NumPy e interpretação de benchmarks

Abordagem	Onde roda o laço principal?	Ponto forte	Cuidado na interpretação
Python puro	Interpretador Python	Didático, explícito	Overhead pode esconder efeitos de memória

NumPy vetorizado	Código compilado	Muito rápido e conciso	Pode usar SIMD/BLAS/threads
Numba @njit	Código nativo gerado	Mantém laços explícitos	Primeira execução inclui compilação
C/C++	Código compilado	Controle fino	Exige toolchain e mais código

12. Medindo consumo de memória

Para um exercício simples, tracemalloc pode monitorar alocações Python. Ele não representa necessariamente toda a memória nativa alocada por bibliotecas externas; por isso, a métrica deve ser descrita corretamente no relatório.

Código 12 — Medição básica de pico de alocação Python

```
import tracemalloc
from time import perf_counter

def medir(func, *args):
    tracemalloc.start()
    t0 = perf_counter()
    result = func(*args)
    dt = perf_counter() - t0
    atual, pico = tracemalloc.get_traced_memory()
    tracemalloc.stop()
    return result, dt, pico / 1024**2

# resultado, tempo, pico_mib = medir(minha_funcao, entrada)
```

13. Desafio 6 — Hierarquia de memória e cache

Comparar diferentes ordens de percurso em matrizes e explicar as diferenças de desempenho com base na organização física da memória e na utilização da cache.

13.1 Experimento obrigatório

- Criar matrizes float64 em C-order com pelo menos cinco tamanhos.
- Implementar soma por linhas e soma por colunas.
- Executar versão Python puro e, preferencialmente, versão Numba.
- Fazer warm-up antes de medir funções JIT.
- Usar pelo menos sete repetições por caso e reportar mediana.
- Confirmar que os resultados numéricos são equivalentes com np.isclose.
- Calcular razão T_colunas / T_linhas.
- Produzir gráfico tempo × dimensão e/ou ns por elemento.
- Consultar o tamanho das caches da máquina usada.
- Relacionar resultados com linha de cache, localidade espacial e conjunto de trabalho.

13.2 Extensão para nível avançado

- Repetir com matriz Fortran-order e prever a inversão do comportamento.
- Testar strides crescentes em um vetor grande.
- Implementar uma operação blocada e variar o tamanho BS.

- Comparar uma versão manual com np.sum ou np.dot, discutindo por que a comparação não isola apenas cache.
- Se o sistema permitir, coletar hardware counters com perf (Linux) e comparar cache-misses/cycles/instructions.

13.3 Estrutura do relatório

- Hardware e software: CPU, RAM, SO, Python, NumPy e Numba.
- Hipótese antes da execução.
- Metodologia de medição.
- Tabela dos resultados.
- Gráficos.
- Interpretação baseada em arquitetura.
- Limitações do experimento.
- Conclusão: qual ordem foi melhor e em quais condições?

14. Roteiro de laboratório em sala

1. Identifique modelo da CPU e tamanhos de L1/L2/L3.
2. Execute o benchmark simples e observe variabilidade entre repetições.
3. Compare as duas versões com cálculo redundante.
4. Execute o experimento de stride e normalize pelo número de acessos.
5. Execute soma por linhas e colunas em Python puro.
6. Repita com Numba e exclua a compilação da medição.
7. Troque C-order por Fortran-order e formule a previsão antes de executar.
8. Varie o tamanho da matriz e procure mudanças no custo por elemento.
9. Discuta quais resultados são explicados por cache e quais podem envolver interpretador, SIMD, prefetch ou bandwidth.

15. Questões conceituais para discussão

10. Duas implementações O(n) precisam ter o mesmo tempo de execução? Justifique.
11. Uma versão com menos operações aritméticas pode ser mais lenta? Dê um cenário relacionado à memória.
12. Por que acessar A[i,j] com j variando rapidamente tende a favorecer uma matriz C-order?
13. Por que uma linha de cache torna a localidade espacial importante?
14. Explique a diferença entre localidade espacial e temporal usando um exemplo de código.
15. Por que uma medição única não é suficiente para comparar implementações?
16. Por que o primeiro tempo de uma função @njit não deve ser comparado diretamente aos demais?
17. Por que np.sum(A) não é uma comparação “pura” entre ordem de percurso?
18. Quando blocking pode melhorar desempenho mesmo sem reduzir a quantidade de operações?
19. Em que sentido o gargalo de memória prepara o caminho para conceitos como roofline e bandwidth?

16. Resultados esperados e pontos de atenção do professor

Experimento	Tendência esperada	Possíveis fatores de confusão
Cálculo redundante	Hoisting reduz tempo	Operação simples demais;

		overhead Python domina
list × set	set muito melhor em muitas consultas	Custo de construir o set; memória adicional
Stride	Custo por acesso tende a piorar com saltos grandes	Prefetch, TLB, número total de acessos
Linha × coluna — Python	Diferença pode ser pequena/ruidosa	Interpretador domina
Linha × coluna — Numba	Linhas geralmente vencem em C-order	Otimizações do compilador, CPU
C-order × F-order	Ordem favorável tende a inverter	Conversão/cópia feita dentro da região medida
Tamanho do working set	Mudanças de regime podem aparecer	Caches compartilhadas, frequência, SO
Blocking	Existe faixa de BS mais eficiente	Python puro é inadequado para medir kernel

17. Conexão com paralelismo e HPC

Esta aula é pré-requisito conceitual para programação paralela. Adicionar núcleos não elimina gargalos de memória. Ao contrário, vários núcleos podem disputar a mesma largura de banda da RAM e caches compartilhadas. Assim, um código sequencial com baixa localidade pode apresentar speedup paralelo ruim mesmo quando há muitas threads disponíveis.

- Amdahl: a parte sequencial limita o speedup máximo.
- Bandwidth: múltiplas threads podem saturar o subsistema de memória.
- False sharing: threads escrevendo em dados distintos da mesma linha de cache podem causar coerência excessiva.
- NUMA: em máquinas maiores, a localização física da memória em relação ao processador importa.
- Blocking: técnica sequencial que também é essencial em kernels paralelos eficientes.

Ideia-chave: Regra prática para a disciplina: primeiro tornar o kernel sequencial correto, mensurável e cache-friendly; depois paralelizar; por fim medir escalabilidade.

18. Checklist de boas práticas

- Defina exatamente o que está sendo medido.
- Valide a correção antes e depois da otimização.
- Use múltiplas repetições e mediana.
- Separe preparação de dados da região cronometrada.
- Não misture compilação JIT com execução.
- Registre hardware e versões do software.
- Compare custo assintótico e custo observado.
- Normalize métricas quando o número de operações/acessos muda.
- Evite atribuir toda diferença de desempenho à cache sem evidência.
- Relate limitações e fontes de ruído.

Apêndice A — Ambiente Python recomendado

Para os exemplos básicos, NumPy é suficiente. Para isolar melhor efeitos de cache em laços explícitos, Numba é recomendado.

Criação sugerida do ambiente Conda

```
conda create -n hpc python=3.11 -y
conda activate hpc
conda install numpy matplotlib numba -y
```

Opcionalmente, psutil pode ser instalado para inspecionar memória do processo; ferramentas de hardware counters dependem do sistema operacional.

Apêndice B — Script-base para os alunos

Base mínima para experimentos reproduzíveis

```
import numpy as np
from time import perf_counter
from statistics import median

def medir(func, *args, repeat=9):
    # Execute warm-up externamente se func usar JIT.
    tempos = []
    ref = None
    for _ in range(repeat):
        t0 = perf_counter()
        value = func(*args)
        tempos.append(perf_counter() - t0)
        if ref is None:
            ref = value
        else:
            assert np.isclose(value, ref)
    return median(tempos), tempos

# 1) Defina as versões
# 2) Faça warm-up quando necessário
# 3) Meça para vários tamanhos
# 4) Armazene resultados
# 5) Gere gráficos e interprete
```

Referências recomendadas

- HENNESSY, J. L.; PATTERSON, D. A. Computer Architecture: A Quantitative Approach. Morgan Kaufmann.
- BRYANT, R. E.; O'HALLARON, D. R. Computer Systems: A Programmer's Perspective. Pearson.
- KIRK, D. B.; HWU, W.-M. W. Programming Massively Parallel Processors. Morgan Kaufmann.
- NumPy Documentation — ndarray memory layout, strides and performance considerations.
- Numba Documentation — just-in-time compilation and performance tips.